Proyecto Serverless Inteligente en AWS

1. Portada

Nombre del proyecto: Serverless Inteligente en AWS
Asignatura: Arquitectura Cloud
Alumno: Christian Cortez
Fecha: 05/04/2026

2. Introducción

En este proyecto se implementa una arquitectura **100% serverless en AWS**, con el objetivo de reducir costos operativos, eliminar la administración de servidores y lograr una solución escalable automáticamente.

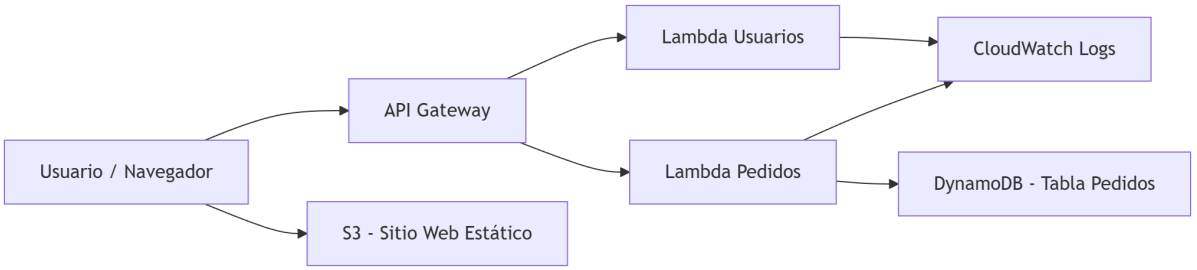
Se utilizan servicios administrados como:

- AWS Lambda
- API Gateway
- DynamoDB
- S3
- CloudWatch

3. Arquitectura de la solución

La arquitectura implementada sigue este flujo:

Usuario → API Gateway → Lambda → DynamoDB / S3



4. Desarrollo por lecciones

Lección 1 – Sitio estático en S3

Se creó un bucket S3 para alojar un sitio web estático.

Pasos realizados:

- Creación del bucket
- Subida de archivo `index.html`
- Activación de hosting estático

Resultado:
El sitio es accesible públicamente mediante la URL del bucket.

Lección 2 – Funciones Lambda

Se implementaron dos funciones:

1. Lambda Usuarios

- Retorna un mensaje simple
- Simula un servicio de usuarios

2. Lambda Pedidos

- Recibe datos de entrada
- Inserta información en DynamoDB

Resultado:
Ambas funciones ejecutan correctamente desde consola.

Lección 3 – API Gateway

Se creó un API HTTP con dos rutas:

- `/usuarios` → Lambda Usuarios
- `/pedidos` → Lambda Pedidos

Resultado:
Los endpoints responden correctamente desde navegador o Postman.

Lección 4 – DynamoDB

Se creó una tabla:

- Nombre: `Pedidos`
- Clave primaria: `orderId`

La Lambda de pedidos inserta registros en la tabla.

Resultado:
Se verifican ítems almacenados correctamente.

Lección 5 – Monitoreo con CloudWatch

Se revisaron los logs de ambas funciones Lambda.

Resultado:
Se evidencian:

- Invocaciones
- Mensajes de ejecución
- Errores (si existen)

5. Validación de requisitos

✔ 2 funciones Lambda funcionando
✔ API Gateway con endpoints activos
✔ DynamoDB con datos insertados
✔ Sitio web en S3 accesible
✔ Logs en CloudWatch visibles
✔ Arquitectura documentada

6. Evidencias

6.1 Lambda ejecutándose

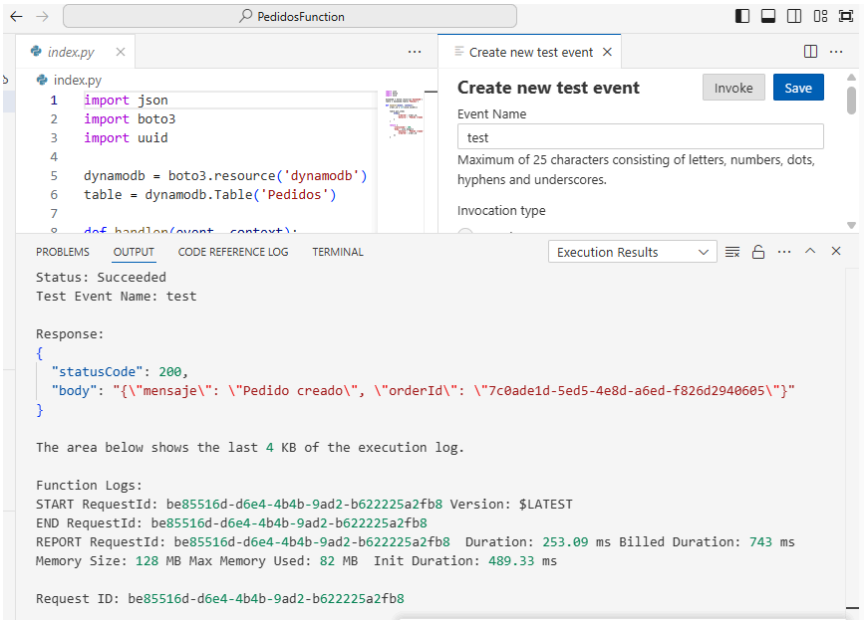
6.1.1 UsuariosFunction

Creamos un test salida {} y lo ejecutamos

```
Status: Succeeded
Test Event Name: test

Response:
{
  "statusCode": 200,
  "body": "Microservicio Usuarios funcionando"
}

The area below shows the last 4 KB of the execution log.

Function Logs:
START RequestId: 3468d07a-cb6b-4d5a-86ed-188f1735120a Version: $LATEST
END RequestId: 3468d07a-cb6b-4d5a-86ed-188f1735120a
REPORT RequestId: 3468d07a-cb6b-4d5a-86ed-188f1735120a  Duration: 1.75 ms   Billed Duration: 106 ms
Memory Size: 128 MB Max Memory Used: 31 MB  Init Duration: 104.00 ms

Request ID: 3468d07a-cb6b-4d5a-86ed-188f1735120a
```

6.1.2 PedidosFunction

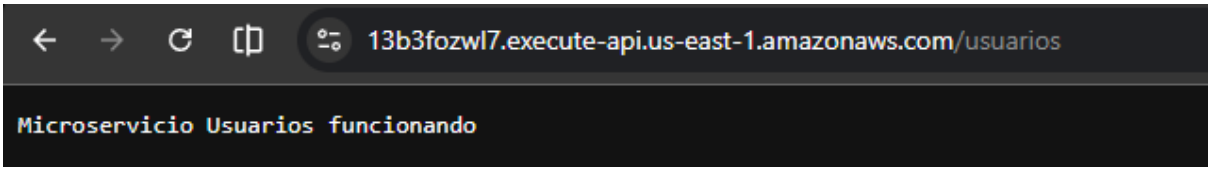
Creamos un test salida {} y lo ejecutamos



6.2 API funcionando (/usuarios y /pedidos)

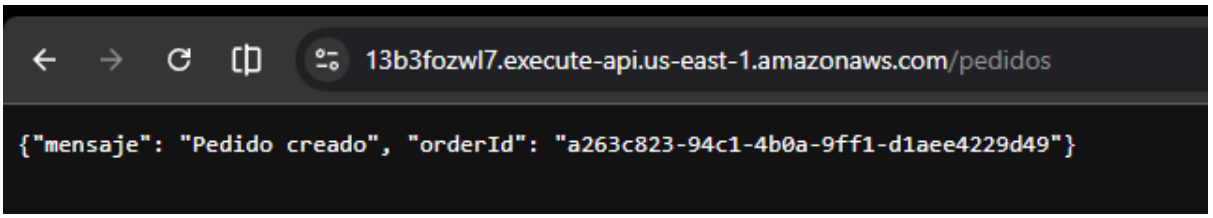
6.2.1 usuarios

http://13b3fozwl7.execute-api.us-east-1.amazonaws.com/usuarios

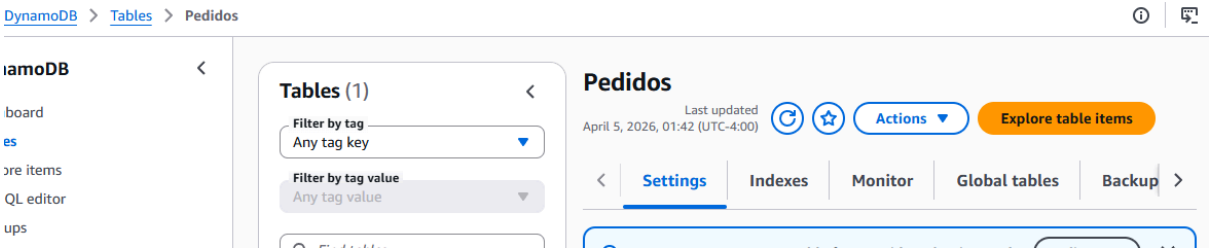


6.2.2 pedidos

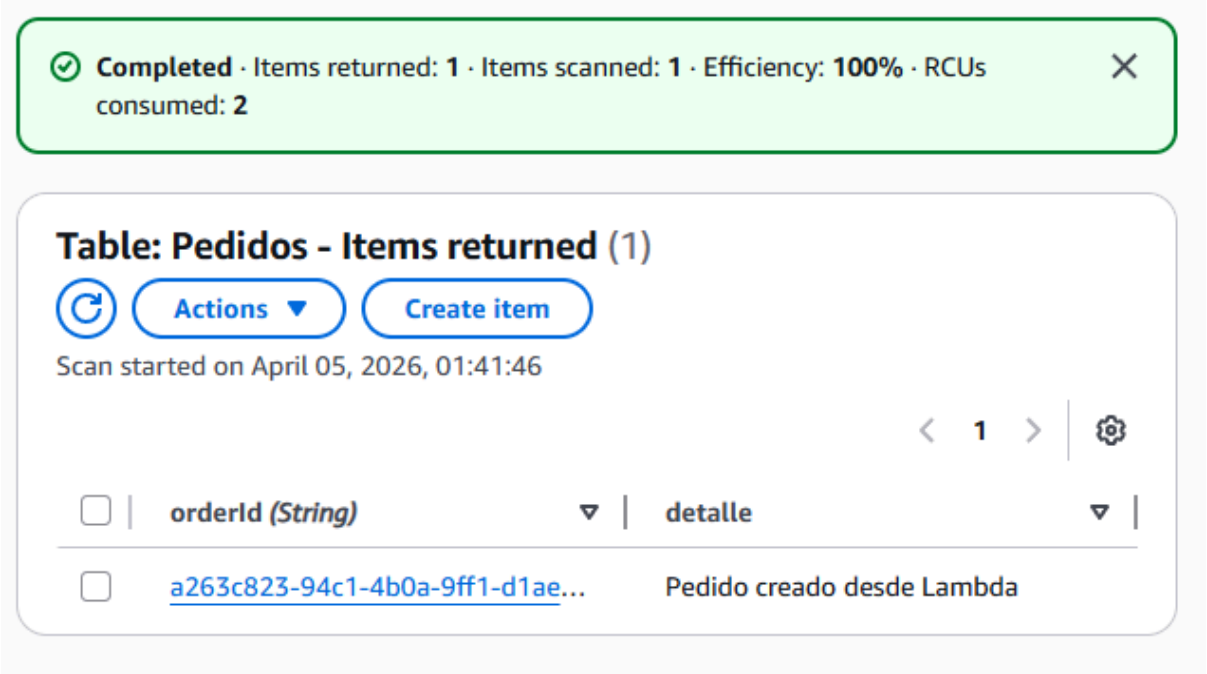
https://13b3fozwl7.execute-api.us-east-1.amazonaws.com/pedidos



6.3 Tabla DynamoDB con datos



6.3.1 Registros

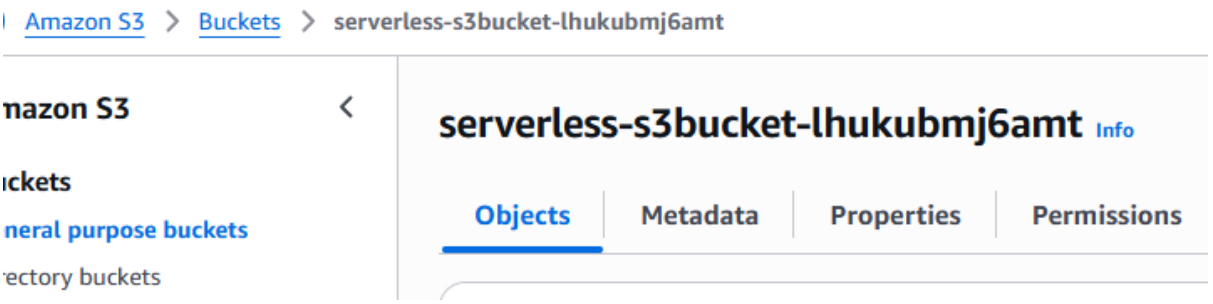


6.4 Sitio web en S3

6.4.1 bucket

le quitamos el bloqueo publico en permissions y agregamos en bucket politic esto

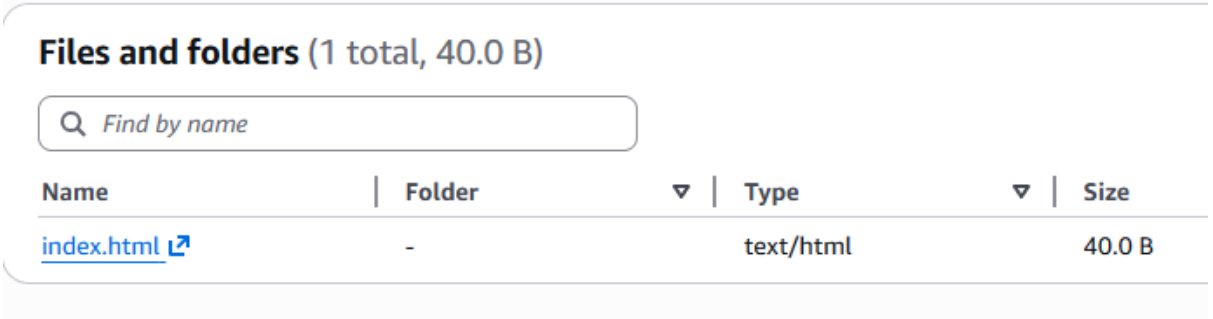
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::serverless-s3bucket-lhukubmj6amt/*"
    }
  ]
}
```



6.4.2 upload

subimos archivo index.html

```
<h1>Proyecto Serverless Inteligente</h1>
```



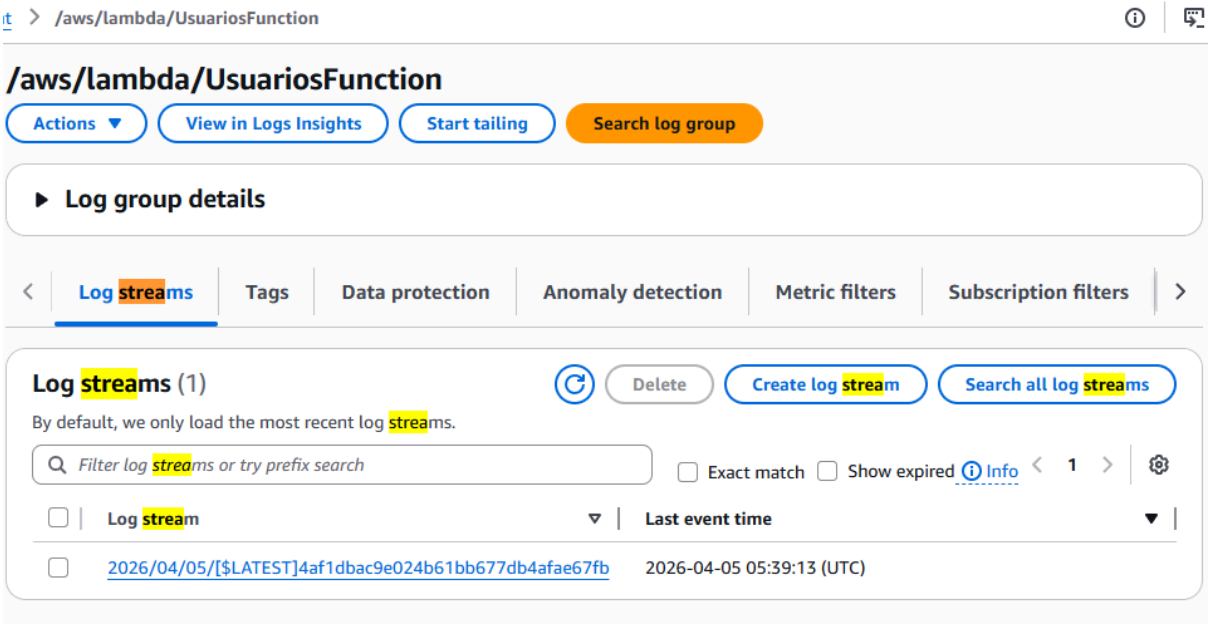
6.4.3 Sitio Web

Nos vamos a Properties → Static website hosting y copiarlos la url

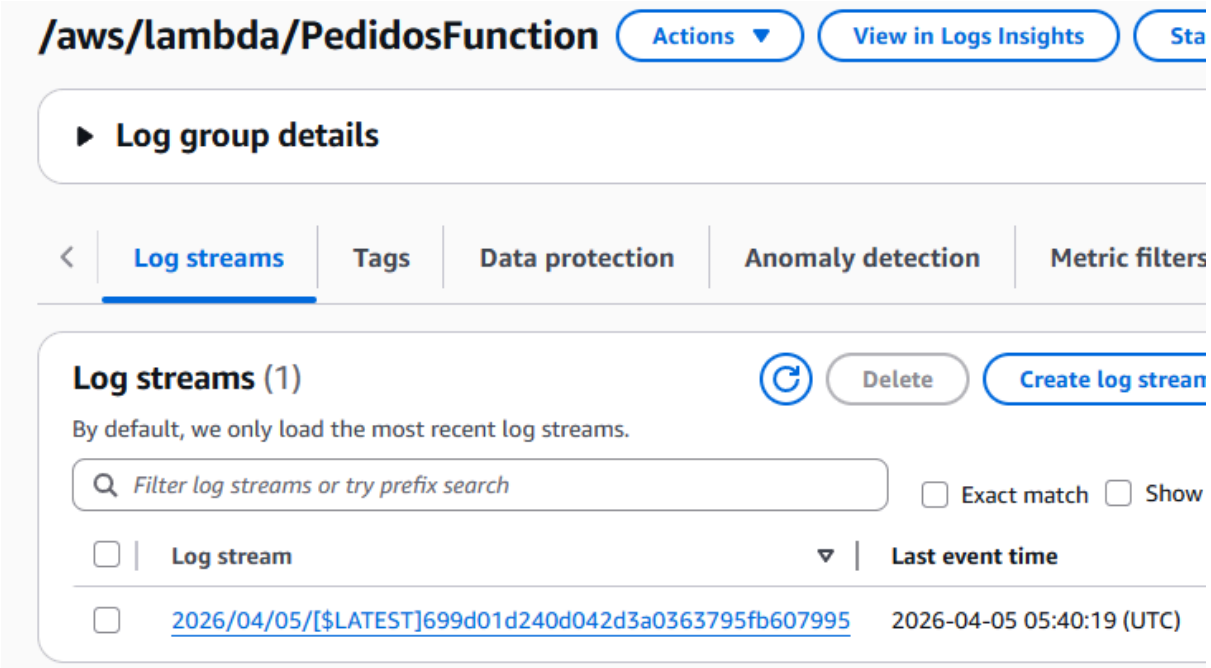
http://serverless-s3bucket-lhukubmj6amt.s3-website-us-east-1.amazonaws.com/

6.5 Logs en CloudWatch

6.5.1 Usuarios



6.5.2 Pedidos



7. Conclusiones

La arquitectura serverless permite:

- Reducir costos (pago por uso)
- Escalar automáticamente
- Eliminar administración de servidores

Además, AWS proporciona integración nativa entre servicios, facilitando el desarrollo de aplicaciones modernas.